

Sparsity Aware Normalization for GANs

Idan Kligvasser, Tomer Michaeli

TechnionIsrael Institute of Technology, Haifa, Israel
{kligvasser@campus, tomer.m@ee}.technion.ac.il

Abstract

Generative adversarial networks (GANs) are known to benefit from regularization or normalization of their critic (discriminator) network during training. In this paper, we analyze the popular spectral normalization scheme, find a significant drawback and introduce *sparsity aware normalization* (SAN), a new alternative approach for stabilizing GAN training. As opposed to other normalization methods, our approach explicitly accounts for the sparse nature of the feature maps in convolutional networks with ReLU activations. We illustrate the effectiveness of our method through extensive experiments with a variety of network architectures. As we show, sparsity is particularly dominant in critics used for image-to-image translation settings. In these cases our approach improves upon existing methods, in less training epochs and with smaller capacity networks, while requiring practically no computational overhead.

1 Introduction

Generative adversarial networks (GANs) (?) have made a dramatic impact on low-level vision and graphics, particularly in tasks relating to image generation (?), image-to-image translation (???), and single image super resolution (???). GANs can generate photo-realistic samples of fantastic quality (????), however they are often hard to train and require careful use of regularization and/or normalization methods for making the training stable and effective.

A factor of key importance in GAN training, is the way by which the critic (discriminator) network is optimized. An overly-sharp discrimination function can lead to gradient vanishing when updating the generator, while an overly-smooth function can lead to poor discrimination between real and fake samples and thus to insufficient supervision for the generator. One of the most successful training approaches, is that arising from the Wasserstein GAN (WGAN) (?) formulation, which asserts that the critic should be chosen among the set of Lipschitz-1 functions. Precisely enforcing this constraint is impractical (?), yet simple approximations, like weight clipping (?) and gradient norm penalty (?), are already quite effective.

Perhaps the most effective approximation strategy is *spectral normalization* (?). This method normalizes the weights

of the critic network after every update step, in an attempt to make each layer Lipschitz-1 individually (which would guarantee that the end-to-end function is Lipschitz-1 as well). Due to its simplicity and its significantly improved results, this approach has become the method of choice in numerous GAN based algorithms (*e.g.* (????)).

In this paper, we present a new weight normalization strategy that outperforms spectral normalization, as well as all other methods, by a significant margin on many tasks and with various network architectures (see *e.g.*, Fig. 1). We start by showing, both theoretically and empirically, that normalizing each layer to be Lipschitz-1 is overly restrictive. In fact, as we illustrate, such a normalization leads to very poor GAN training if done correctly. We identify that the real reason for the success of (?) is actually its systematic bias in the estimation of the Lipschitz constant for convolution layers, which is typically off by roughly a factor of 4. Following our analysis, we show that a better way to control the end-to-end smoothness of the critic, is to normalize each layer by its amplification of the *typical signals* that enter it (rather than the worst-case ones). As we demonstrate, in convolutional networks with ReLU activations, these signals are typically channel-sparse (namely many of their channels are identically zero). This motivates us to suggest *sparsity aware normalization* (SAN).

Our normalization has several advantages over spectral normalization. First, it leads to better visual results, as also supported by quantitative evaluations with the Inception score (IS) (?) and the Fréchet Inception distance (FID) (?). This is true in both unconditional image generation and conditional tasks, such as label-to-image translation, super-resolution, and attribute transfer. Second, our approach better stabilizes the training, and it does so at practically no computational overhead. In particular, even if we apply only a single update step of the critic for each update of the generator, and normalize its weights only once every 1K steps, we still obtain an improvement over spectral normalization. Finally, while spectral normalization benefits from different tuning of the optimization hyper-parameters for different tasks, our approach works well with the precise same settings for all tasks.

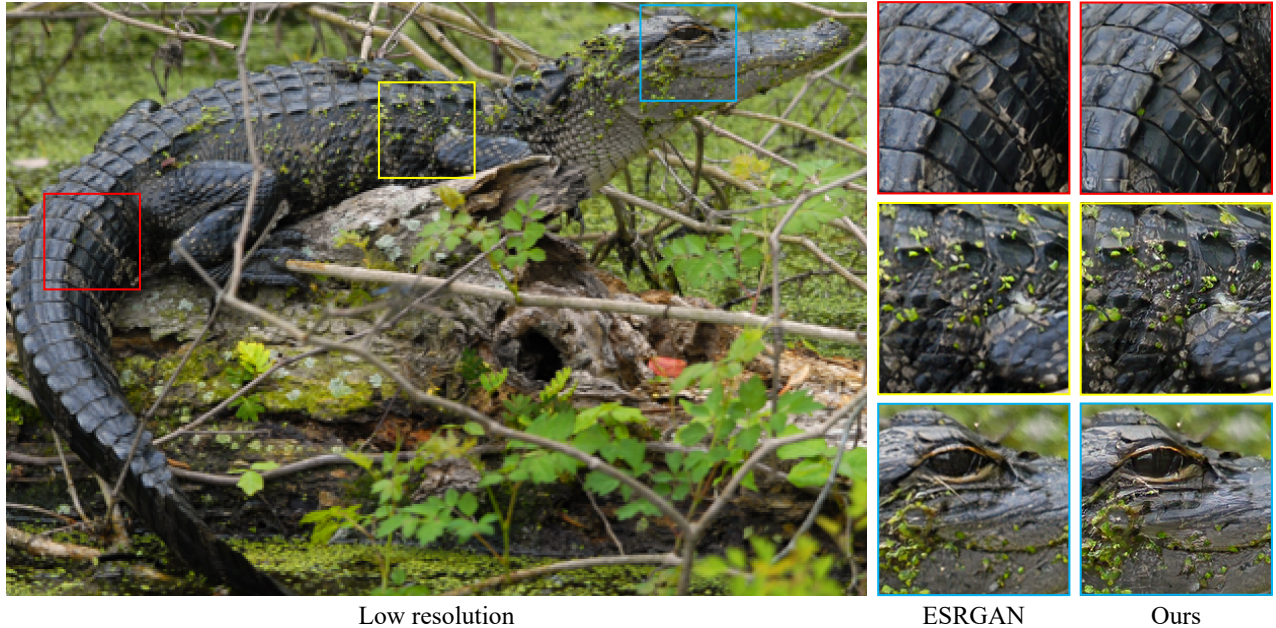


Figure 1: **Super resolution with our sparsity aware normalization.** Our technique can boost the performance of any GAN-based method, while allowing less training epochs and smaller models. For example, in the task of $4\times$ super-resolution, we achieve more photo-realistic reconstructions than the state-of-the-art ESRGAN network (?), while using a model with only 9% the number of parameters of ESRGAN (1.5M for ours and 16.7M for ESRGAN).

2 Rethinking Per-Layer Normalization

GANs (?) minimize the distance between the distribution of their generated “fake” samples, $\mathbb{P}_F$, and the distribution of real images, $\mathbb{P}_R$, by diminishing the ability to discriminate between samples drawn from $\mathbb{P}_F$ and samples drawn from $\mathbb{P}_R$. In particular, the Wasserstein GAN (?) targets the minimization of the Wasserstein distance between $\mathbb{P}_F$ and $\mathbb{P}_R$, which can be expressed as

$$W(\mathbb{P}_R, \mathbb{P}_F) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_R}[f(x)] - \mathbb{E}_{x \sim \mathbb{P}_F}[f(x)]. \quad (1)$$

Here, the optimization is over all critic functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$ whose Lipschitz constant is no larger than 1. Thus, the critic’s goal is to output large values for samples from $\mathbb{P}_R$ and small values for samples from $\mathbb{P}_F$. The GAN’s generator attempts to shape the distribution of fake samples, $\mathbb{P}_F$, so as to minimize $W(\mathbb{P}_R, \mathbb{P}_F)$ and so to rather decrease this gap.

The Lipschitz constraint has an important role in the training of WGANs, as it prevents overly sharp discrimination functions that hinder the ability to update the generator. However, since f is a neural network, this constraint is impractical to enforce precisely (?), and existing methods resort to rather inaccurate approximations. Perhaps the simplest approach is to clip the weights of the critic network (?). However, this leads to stability issues if the clipping value is taken to be too small or too large. An alternative, is to penalize the norm of the gradient of the critic network (?). Yet, this often has poor generalization to points outside the support of the current generative distribution.

To mitigate these problems, Miyato *et al.* (?) suggested to enforce the Lipschitz constraint on each layer individually.

Specifically, denoting the function applied by the i th layer by $\phi_i(\cdot)$, we can write

$$f(x) = (\phi_N \circ \phi_{N-1} \circ \dots \circ \phi_1)(x). \quad (2)$$

Now, since $\|\phi_1 \circ \phi_2\|_L \leq \|\phi_1\|_L \cdot \|\phi_2\|_L$, we have that

$$\|f\|_L \leq \|\phi_N\|_L \cdot \|\phi_{N-1}\|_L \cdot \dots \cdot \|\phi_1\|_L. \quad (3)$$

This implies that restricting each ϕ_i to be Lipschitz-1, ensures that f is also Lipschitz-1. Popular activation functions, such as ReLU and leaky ReLU, are Lipschitz-1 by construction. For linear layers (like convolutions), ensuring the Lipschitz condition merely requires normalizing the weights by the Lipschitz constant of the transform, which is the top singular value of the corresponding weight matrix.

This per-layer normalization strategy has gained significant popularity due to its simplicity and the improved results that it provides when compared to the preceding alternatives. However, close inspection reveals that normalizing each layer by its top singular value is actually too conservative. That is, restricting each layer to be Lipschitz-1, typically leads to a *much smaller set of permissible functions* than the set of functions whose end-to-end Lipschitz constant is 1. As a simple illustration, consider the following example (see Appendix A).

Example 1 Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a two-layer network with

$$\phi_1(x) = \sigma(w_1 x + b_1), \quad \phi_2(z) = w_2^T z + b_2, \quad (4)$$

where σ is the ReLU activation function, $w_1, w_2, b_1 \in \mathbb{R}^n$, and $b_2 \in \mathbb{R}$. Such a critic can implement any continuous

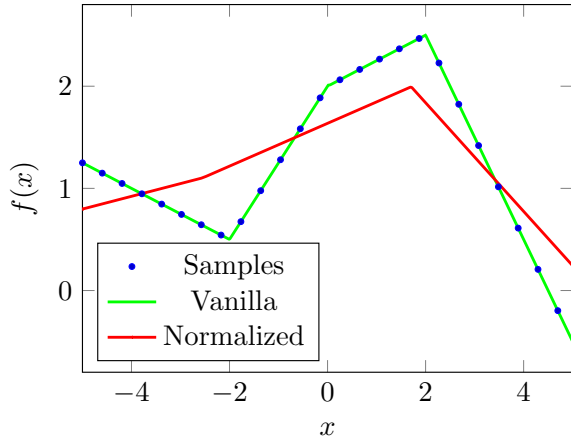


Figure 2: **Fitting to a Lipschitz-1 function.** Here, we trained a network with one hidden layer to fit to samples of a Lipschitz-1 function (blue dots). When using vanilla training without any normalization, the fit is perfect (green). However, when using layer-wise spectral normalization, the fit is poor (red). This illustrates the fact that the set of functions that can be represented by a network with Lipschitz-1 layers is often significantly smaller than the set of all Lipschitz-1 functions that can be represented by the same architecture.

piece-wise linear function with $n+1$ segments. Now, the end-to-end constraint $\|f\|_L \leq 1$, restricts the slope of each segment to satisfy $|f'(x)| \leq 1$. But the layer-wise constraints¹ $\|w_1\| \leq 1$, $\|w_2\| \leq 1$, allow a much smaller set of functions, as they also impose for example that $|f'(-\infty) + f'(\infty)| \leq 1$. In particular, they rule out the identity function $f(x) = x$, and also any function with slope larger than 0.5 or smaller than -0.5 simultaneously for $x \rightarrow \infty$ and $x \rightarrow -\infty$. This is illustrated in Fig. 2.

This example highlights an important point. When we normalize a layer by its top singular value, we restrict how much it can amplify an arbitrary input. However, this is overly pessimistic since not all inputs to that layer are admissible. In the example above, for most choices of w_1 the input to the second layer is necessarily sparse because of the ReLU. Specifically, if $b_1 = 0$ and w_1 has k_p positive entries and k_n negative ones, then the output of the first layer cannot contain more than $\max\{k_p, k_n\}$ non-zero entries. This suggests that when normalizing the second layer, we should only consider how much it amplifies sparse vectors.

As a network gets deeper, the attenuation caused by such layer-wise normalization accumulates, and severely impairs the network’s representation power. One may wonder, then, why the layer-wise spectral normalization of (?) works in practice after all. The answer is that for convolutional layers, this method uses a very crude approximation of the top singular value, which is typically $4\times$ smaller than the true top singular value². We empirically illustrate this in Fig. 3

¹Since w_1 and w_2 are $n \times 1$, their top singular value is simply their Euclidean norm.

²In (?), the top singular value of the convolution operation is

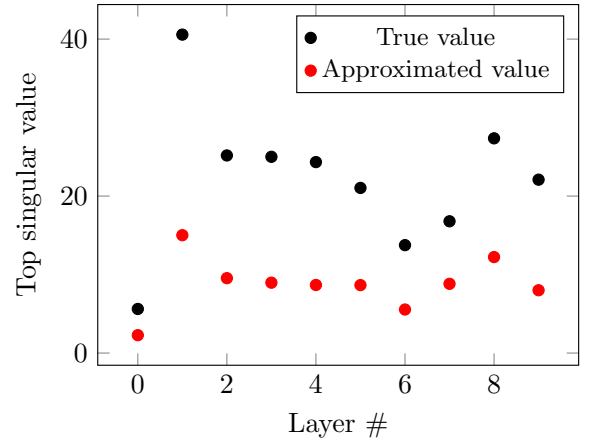


Figure 3: **Top singular value.** We plot the top singular value (black) of each convolution layer of a trained ResNet critic network, as well as its approximation employed by (?) (red). The approximation is typically much smaller than the actual value, implying that the weights after normalization are in fact much larger than intended.

for a ResNet critic architecture, where we use the Fourier domain formulation of (?) to compute the true top singular value. This observation implies that in (?), the weights after normalization are in fact much larger than intended.

What would happen had we normalized each layer by its true top singular value? As shown in Fig. 4, in this case, the training completely fails. This is because the weights become extremely small and the gradients vanish.

3 Sparsity Aware Normalization

We saw that the spectral normalization of (?) is effective because of the particular approximation used for $\|\phi_i\|_L$. A natural question, then, is whether we can somehow improve upon this normalization scheme. A naive approach would be to set a multiplier parameter σ , to adjust their normalization constant. However, as the authors of (?) themselves indicate, such a parameter does not improve their results. This implies that the set of discriminator functions satisfying their per-layer constraints does not overlap well with the set of Lipschitz-1 functions as neither dilation nor erosion of this set improves their results.

A more appropriate strategy is therefore to seek for a normalization method that explicitly accounts for the statistics of signals that enter each layer. An important observation in this respect, is that in convolutional networks with ReLU activations, the features are typically channel-sparse. That is, for most input signals, many of the channels are identically zero. This is illustrated in Fig. 5, which shows a histogram of the norms of the channels of the last layer of a trained critic³, computed over 2048 randomly sampled images from the training set.

approximated by the top singular value of a 2D matrix obtained by reshaping the 4D kernel tensor.

³We used no normalization, but chose a run that converged.



Figure 4: **The effect of normalization.** Here, we trained a WGAN on the CIFAR-10 dataset using exact per-layer spectral normalization and the approximate method of (?), both initialized with the same draw of random weights. The exact normalization does not converge, while the approximate one rather leads to good conditioning.

In light of this observation, rather than normalizing a layer $\phi(x) = Wx + b$ by its Lipschitz constant,

$$\|\phi\|_L = \sup_{\|x\| \leq 1} \|Wx\|, \quad (5)$$

here we propose to modify the constraint set to take into account only channel-sparse signals. Moreover, since we know that many output channels are going to be zeroed out by the ReLU that follows, we also modify the objective of (5) to consider the norm of each output channel individually.

Concretely, for a multi-channel signal x with channels $x_1, \dots, x_k$, let us denote by $\|x\|_\infty$ its largest channel norm, $\max\{\|x_i\|\}$, and by $\|x\|_0$ its number of nonzero channels, $\#\{\|x_i\| > 0\}$. With these definitions, we take our normalization constant to be⁴

$$\|W\|_{0,\infty} \triangleq \sup_{\substack{\|x\|_0 \leq 1 \\ \|x\|_\infty \leq 1}} \|Wx\|_\infty. \quad (6)$$

Normalizing by $\|W\|_{0,\infty}$ ensures that there exists no 1-sparse input signal (*i.e.* with a single nonzero channel) that can cause the norm of some output channel to exceed 1.

For convolutional layers, computing $\|W\|_{0,\infty}$ is simple. Specifically, if W has n input channels and m output channels, then the i th channel of $y = Wx$ can be expressed as

$$y_i = \sum_{j=1}^n w_{i,j} * x_j. \quad (7)$$

Here, ‘ $*$ ’ denotes single-input-single-output convolution and $w_{i,j}$ is the kernel that links input channel j with output channel i . Now, using the kernels $\{w_{i,j}\}$, we can compute $\|W\|_{0,\infty}$ as follows (see Appendix B).

Lemma 1 *For a multiple-input-multiple-output filter W with cyclic padding,*

$$\|W\|_{0,\infty} = \max_{i,j} \|\mathcal{F}\{w_{i,j}\}\|_\infty, \quad (8)$$

⁴Note that $\|\cdot\|_{0,\infty}$ is not a norm since ℓ_0 is not a norm.

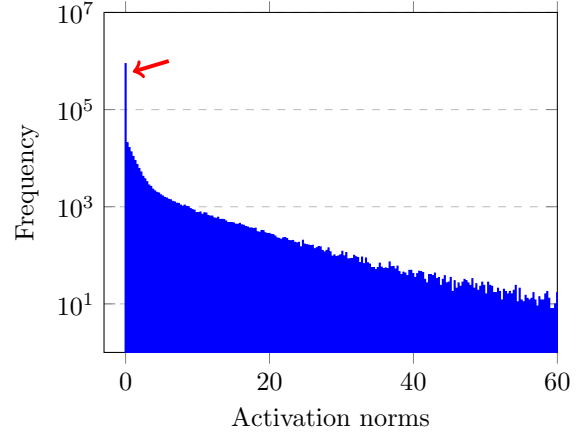


Figure 5: **Channel sparsity.** The histogram of the channel norms at the last layer of a critic trained without normalization. For most input images, many of the channels are identically zero.

where $\mathcal{F}\{w_{i,j}\}$ is the discrete Fourier transform of $w_{i,j}$, zero-padded to the spatial dimensions of the channels.

Thus, to compute our normalization constant, all we need to do is take the Fourier transform of each kernel, find the maximal absolute value in the transform domain, and then take the largest among these $m \times n$ top Fourier values.

3.1 Efficiency

To take advantage of Lemma 1, we use cyclic padding for all convolutional layers of the critic. This allows us to employ the fast Fourier transform (FFT) for computing the normalization constants of the layers. For fully-connected layers, we use the top singular value of the eight matrix, as in (?). The overhead in running time is negligible. For example, on CIFAR-10, each critic update takes the same time as spectral normalization and 20% less than gradient-penalty regularization (see Appendix F).

In models for large images, storing the FFTs of all the filters of a layer can be prohibitive. In such settings, we compute the maximum in (8) only over a random subset of the filters. We compensate for our under-estimation of the maximum by multiplying the resulting value by a scalar g . As we show in the Appendix, the optimal value of g varies very slowly as a function of the percentage of chosen filters (*e.g.* it typically does not exceed 1.3 even for ratios as low as 25%). This can be understood by regarding the kernels’ top Fourier coefficients as independent draws from some density. When this density decays fast, the expected value of the maximum over k draws increases very slowly for large k . For example, for the exponential distribution (which we find to be a good approximation), we show in the Appendix that the optimal g for ratio r is given by

$$g = \frac{\sum_{j=1}^{m \cdot n} \frac{1}{m \cdot n - j + 1}}{\sum_{j=1}^{\lceil m \cdot n \cdot r \rceil} \frac{1}{\lceil m \cdot n \cdot r \rceil - j + 1}}, \quad (9)$$

leading to *e.g.* $g \approx 1.2$ for $r = 25\%$ with $m \cdot n = 64^2$ filters.

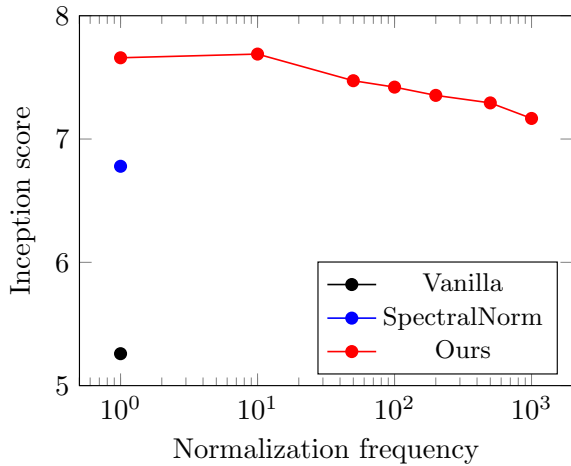


Figure 6: **Efficiency.** Here we compare three WGANs, trained for 100 epochs on the CIFAR-10 dataset (?): (i) without normalization, (ii) with spectral-normalization (?), (iii) with our normalization. The training configurations and the initial seed are the same for all networks. In contrast to (?), which performs weight normalization after each critic update, in our method we can normalize the layers much less frequently. Pay attention that even if we normalize only once every 1000 steps, less than 80 updates in total, we still outperform spectral normalization by a large margin.

The effect of our normalization turns out to be very strong and therefore, besides a boost in performance, it also allows more efficient training than spectral normalization (?) and other WGAN methods. Particularly:

Critic updates: For every update step of the generator, we perform only one update step of the critic. This is in contrast to other WGAN schemes, which typically use at least three (???). Despite the fewer updates, our method converges faster (see Appendix E).

Normalization frequency: In spectral normalization, the weights are normalized after each critic update (using a single iteration of the power method). In contrast, we can normalize the layers much less frequently and still obtain a boost in performance. For example, as shown in Fig. 6, even if we normalize only once every 1000 steps, we still outperform spectral normalization by a large margin.

Hyper-parameters: As opposed to other normalization methods, like (?????), our algorithm does not require special hyper-parameter tuning for different tasks. All our experiments use the same hyper-parameters.

4 Experiments

We now demonstrate the effectiveness of our approach in several tasks. In all our experiments, we apply normalization after each critic update step to obtain the best results.

4.1 Image Generation

We start by performing image generation experiments on the CIFAR-10 (?) and STL-10 (?) datasets. We use these simple

Method	CIFAR-10	STL-10
Real-data	11.24 ± .12	26.08 ± .26
Unconditional GAN (Standard CNN)		
Weight clipping	6.41 ± .11	7.57 ± .10
WGAN-GP	6.68 ± .06	8.42 ± .13
Batch norm	6.27 ± .10	-
Layer norm	7.19 ± .12	7.61 ± .12
Weight norm	6.84 ± .07	7.16 ± .10
Orthonormal	7.40 ± .12	8.67 ± .08
SN-GANs	7.58 ± .12	8.79 ± .14
(ours) SAN-GANs	7.89 ± .09	9.18 ± .06
Unconditional GAN (ResNet)		
Orthonormal	7.92 ± .04	8.72 ± .06
SN-GANs	8.22 ± .05	9.10 ± .04
(ours) SAN-GANs	8.43 ± .13	9.21 ± .10
Conditional GAN (ResNet)		
BigGAN	9.24 ± .16	-
(ours) SAN-BigGAN	9.53 ± .13	-

Table 1: Inception scores for image generation on the CIFAR-10 and STL-10 datasets. The SAN method outperforms all other regularization methods.

test-beds only for the purpose of comparing different regularization methods on the same architectures. Here, we use $r = 100\%$ of the filters (and thus a compensation of $g = 1$).

Our first set of architectures is that used in (?). But to showcase the effectiveness of our method, in our STL-10 ResNet critic we remove the last residual block, which cuts its number of parameters by 75%, from 19.5M to 4.8M (the competing methods use the 19.5M variant). The architectures are described in full in the Appendix. As in (?), we use the hinge loss (?) for the critic’s updates. We train all networks for 200 epochs with batches of 64 using the Adam optimizer (?). We use a learning rate of $2 \cdot 10^{-4}$ and momentum parameters $\beta_1 = 0.5$ and $\beta_2 = 0.9$.

Additionally, we experiment with the more modern BigGAN architecture (?) for conditional generation on CIFAR-10. We replace the spectral normalization by our SAN in all critic’s res-blocks, and modify Adam’s first momentum parameter to $\beta_1 = 0.5$.

Table 1 shows comparisons between our approach (SAN-GAN) and other regularization methods in terms of Inception score (?). The competing methods include weight clipping (?), gradient penalty (WGAN-GP) (?), batch norm (?), layer norm (?), weight norm (?), orthonormal regularization (?), and spectral normalization (SN-GAN) (?). As can be seen, our models outperform the others by a large gap. Most notably, SAN-BigGAN performs substantially better than the original BigGAN, and sets a new state-of-the-art in conditional image generation on CIFAR-10.

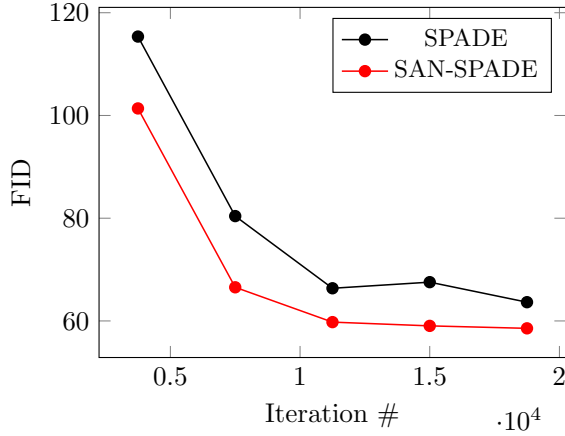


Figure 7: **Model convergence in image translation.** The FID score of our SAN-SPADE converges faster and to a better result than the original SPADE.



Figure 8: **Visual comparison for image translation.** The images synthesized by our SAN-SPADE have less artifacts and contain more fine details than the original SPADE.

4.2 Image-to-Image Translation

Next, we illustrate our method in the challenging task of translating images between different domains. Here we focus on converting semantic segmentation masks to photo-realistic images. In the Appendix, we also demonstrate the power of SAN for attribute transfer.

We adopt the state-of-the-art SPADE scheme (?) as a baseline framework, and enhance its results by applying our normalization. We use the same multi-scale discriminator as (?), except that we replace the zero padding by circular padding and preform SAN. To reduce the memory footprint, we use $r = 25\%$ of the filters with a compensation factor of $g = 1.3$. All hyper-parameters are kept as in (?), except for Adam’s first momentum parameter, which we set to $\beta_1 = 0.5$.

We use 512×256 images from the Cityscapes dataset (?). For quantitative evaluation, we use the Fréchet Inception distance (FID). As can be seen in Fig. 7, our method converges faster and leads to a final model that outperforms the original SPADE by a non-negligible margin. Specifically, SAN-SPADE achieves an FID of 58.56 while the original SPADE

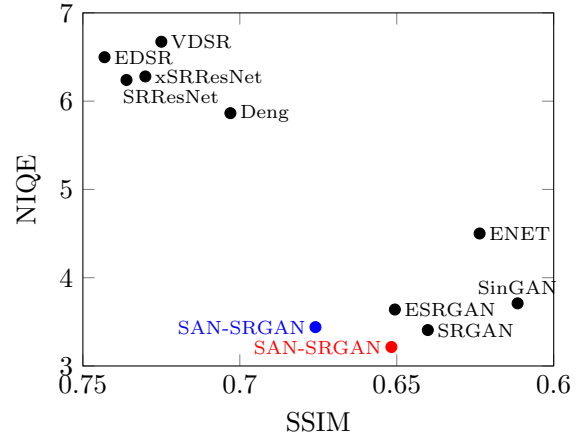


Figure 9: **Perception-distortion evaluation for SR.** We compare our models ($\lambda_{\text{adversarial}} = 10^{-1}$ and 10^{-2}) to other state-of-the-art super-resolution models in terms of perceptual quality (NIQE, lower is better) and distortion (SSIM). Our method improves upon all existing perceptual methods (those at the bottom right) in both perceptual quality and distortion.

Method	BSD100	URBAN100	DIV2K
SRGAN	25.18 / 3.40		
ENET	24.93 / 4.52	23.54 / 3.79	
ESRGAN	25.31 / 3.64	24.36 / 4.23	28.18 / 3.14
ESRGAN*	25.69 / 3.56	24.36 / 3.96	28.22 / 3.06
Ours (0.1)	25.32 / 3.21	23.86 / 3.70	27.74 / 2.87
Ours (0.01)	26.15 / 3.44	24.85 / 3.83	28.76 / 3.16

Table 2: PSNR/NIQE comparison among different perceptual SR methods on varied datasets. Our models attain a significantly higher PSNR and lower NIQE than other perceptual SR methods.

achieves 63.65. Figure 8 shows a qualitative comparison between SPADE and our SAN version after 1.1×10^4 iterations. As can be seen, our synthesized images have less artifacts and contain more details.

4.3 Single Image Super Resolution

Finally, we illustrate SAN in single image super resolution (SR), where the goal is to restore a high resolution image from its down-sampled low resolution version. We focus on $4\times$ SR for images down-sampled with a bicubic kernel.

Following the state-of-the-art ESRGAN (?) method, our loss function comprises three terms,

$$\mathcal{L} = \lambda_{\text{content}} \cdot \mathcal{L}_{\text{content}} + \mathcal{L}_{\text{features}} + \lambda_{\text{adversarial}} \cdot \mathcal{L}_{\text{adversarial}}. \quad (10)$$

Here, $\mathcal{L}_{\text{content}}$ is the L_1 distance between the reconstructed high-res image $\hat{x}$ and the ground truth image x . The term $\mathcal{L}_{\text{features}}$ measures the distance between the deep features of $\hat{x}$ and x , taken from 4th convolution layer (before the 5th max-pooling) of a pre-trained 19-layer VGG network (?). Lastly, $\mathcal{L}_{\text{adversarial}}$ is an adversarial loss that encourages the restored

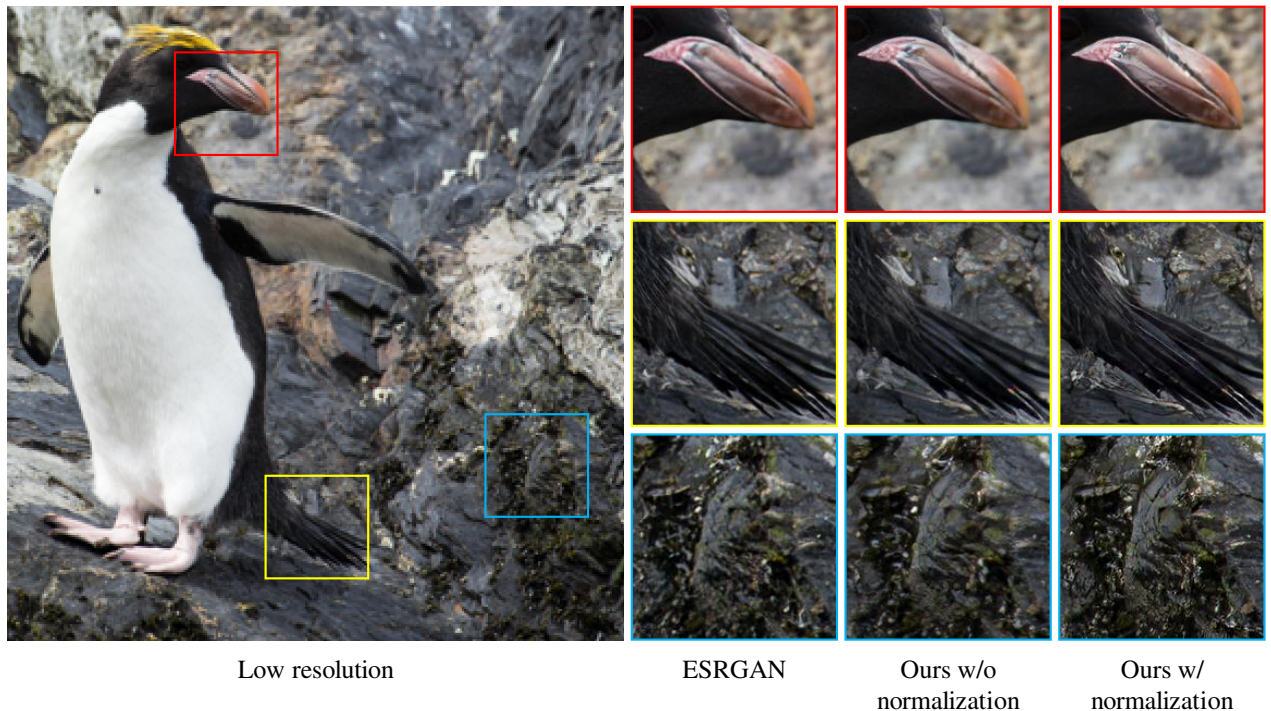


Figure 10: **The influence of normalization in super-resolution.** We compare the state-of-the-art ESRGAN method to our approach, with and without normalization, at a magnification factor of $4\times$. As can be seen, our normalization leads to sharper and more photo-realistic images.

images to follow the statistics of natural images. Here, we use again the hinge loss.

For the generator, we use the much slimmer SRGAN network (?), so that our model has only 9% the number of parameters of ESRGAN (1.5M for ours and 16.7M for ESRGAN). As suggested in (?), we remove the batch normalization layers from the generator. For the critic network, we choose a simple feed forward CNN architecture with 10 convolutional layers and 2 fully connected ones (see Appendix J).

We train our network using the 800 DIV2K training images (?), enriched by random cropping and horizontal flipping. The generator’s weights are initialized to those of a pre-trained model optimized to minimize mean squared error. We minimize the loss (10) with $\lambda_{\text{content}} = 10^{-2}$, and for the adversarial term, $\lambda_{\text{adversarial}}$ we examine two options of 10^{-1} and 10^{-2} . We use the Adam optimizer (?) with momentum parameters set to 0.5 and 0.9, as in Section 4.1. We use a batch size of 32 for 400K equal discriminator and generator updates. The learning rate is initialized to $2 \cdot 10^{-4}$ and is decreased by a factor of 2 at 12.5%, 25%, 50% and 75% of the total number of iterations.

Following (?), we compare our method to other super-resolution schemes in terms of both perceptual quality and distortion. Figure 9 shows a comparison against EDSR (?), VDSR (?), SRResNet (?), xSRResNet (?), Deng (?), ESRGAN(?), SRGAN (?), ENET (?) and SinGAN (?). Here, perceptual quality is quantified using the no-reference NIQE

metric (?) (lower is better), which has been found in (?) to correlate well with human opinion scores in this task. Distortion is measured by SSIM (?) (higher is better). We report average scores over the BSD100 test set (?). As can be seen, our method achieves the best perceptual quality, and lower distortion levels than the perceptual methods (at the bottom right).

In Table 2, we report comparisons to the best perceptual methods on two more datasets, the URBAN100 and DIV2K test sets. As the original ESRGAN (?) uses gradient penalty as a normalization scheme, for a fair comparison, we train an equivalent version, ESRGAN*, with spectral normalization (?). Note that our model outperforms ESRGAN (winner of the PRIM challenge on perceptual image super-resolution (?)) as well as the improved variant ESRGAN*. This is despite the fact that our generator network has only 9% the number of parameters of ESRGAN’s generator. Furthermore, while our model has the same generator architecture as SRGAN, it outperforms it by 1dB in PSNR without any sacrifice in perceptual score.

Figures 1 and 11 shows a visual comparison with ESRGAN. As can be seen, our method manages to restore more of the fine image details, and produces more realistic textures. Figure 10 shows yet another visual result, where we specifically illustrate the effect of our normalization. While without normalization our method is slightly inferior to ESRGAN, when we incorporate our normalization, the visual quality is significantly improved.



Figure 11: **Further super-resolution comparisons.** Compared to ESRGAN, our method better recovers textures, like grass and stones.

4.4 Limitations

SAN does not provide a boost in performance when the critic’s feature maps do not exhibit strong channel-sparsity. This happens, for example, in BigGAN for 128×128 images (see Appendix H). There, there is one set of features in each res-block that are less sparse (those after the residual connection). A possible solution could be to use a different compensation factor g for different layers, according to their level or sparsity. However, we leave this for future work.

5 Conclusion

We presented a new per-layer normalization method for GANs, which explicitly accounts for the statistics of signals that enter each layer. We showed that this approach stabilizes the training and leads to improved results over other GAN schemes. Our normalization adds a marginal computational burden compared to the forward and backward passes, and can even be applied once every several hundred steps while still providing a significant benefit.

Acknowledgements This research was supported by the Israel Science Foundation (grant 852/17) and by the Technion Ollendorff Minerva Center.

Eos aut in qui necessitatibus aliquam asperiores voluptates, consectetur odio alias sit laborum voluptas, cumque eligendi hic dolorem iste beatae nihil numquam architecto tenetur assumenda sint. Delectus sapiente repudiandae id magni nisi incidunt preferendis ipsa, maxime optio exercitationem modi porro reiciendis aliquid sunt obcaecati possumus, reiciendis delectus veniam sapiente non, dolorem la-

bore quis deserunt rem illum laborum impedit quasi magni necessitatibus ut. Vitae obcaecati veritatis laborum pariatur veniam amet dolores doloribus animi dicta cupiditate, fuga pariatur est numquam sequi ipsam explicabo aut accusamus, magni unde eos quo assumenda voluptas aperiam culpa expedita corrupti, id rerum voluptatum aspernatur dolore quibusdam? Porro eos ipsam dolorum quos voluptates delectus tempora quasi consectetur, doloremque eligendi maxime fuga iusto optio aliquam voluptatum iure perspiciatis non, aspernatur odio quam ex labore quo dolorum veritatis commodi, nobis amet eaque nemo cum suscipit. Adipisci ea voluptatem sit ipsum blanditiis, quis ipsam distinctio sit earum eum labore adipisci ipsum, cumque ullam consequatur mollitia? Officiis minima natus velit, nisi facere nulla quasi culpa dolorem? Eligendi suscipit deserunt vitae obcaecati eaque nulla excepturi iste tempore, sit eius veniam optio minus est repudiandae tempore distinctio? Itaque sed culpa exercitationem deserunt earum accusamus veniam illo soluta perspiciatis cum, id laudantium alias aperiam labore necessitatibus autem, nulla provident aliquam commodi minima rem, quis beatae officiis quo recusandae voluptate quos maxime ipsum, numquam quia deleniti atque consequatur expedita soluta reprehenderit ad aut. Dolorem sit minima quidem iusto doloribus itaque ab eius, voluptatem cumque neque quibusdam, soluta dolorem quisquam, in qui dolor ut asperiores quisquam blanditiis iusto delectus vel nesciunt tempora. Repellat explicabo nesciunt excepturi incidunt expedita, itaque aliquam nesciunt, voluptate impedit adipisci mollitia facere autem odit enim aut corporis, animi odio quas. Vero nulla ducimus optio nisi ipsam voluptatum sapi-

ente itaque culpa, deleniti doloribus totam quisquam eveniet
quo earum accusantium quam sit quasi nemo, officia fugit
exercitationem corrupti assumenda?